TP5 - Intro à la statistique spatiale 2A - PostGIS - corrigé

OBJECTIFS DU TP :

- Le but de ce TP est de découvrir l'utilisation de PostGis (extension PostGreSQL) pour la manipulation d'objets spaciaux.
- Afin d'utiliser une version de R plus récente (et une version des packages les plus récentes aussi), vous travaillerez sur le datalab (plateforme du sspcloud, service de l'Insee) : https://datalab.sspcloud.fr.

Les packages nécessaires pour ce TP sont listés ci-dessous :

```
# Chargement des packages
library(dplyr)
library(sf)
library(leaflet)
library(DBI)
library(dbplyr)
# install.packages("osrm")
# install.packages("RPostgres")
library(RPostgres)
```

Les données seront directement téléchargées depuis votre programme R.

Exercice 1 - Création d'une BDD PostGre avec extension PostGIS

0. Comme pour chaque séance, commencer par créer un projet. Déposer ensuite au sein de votre projet les programmes `connexion_db.R` et `create_db.R`.
1. Au sein du datalab, vous allez créer un service PostGre avec extension PostGIS afin d'y stocker ultérieurement vos données. Pour cela, une fois connecté à la plateforme du datalab : aller dans **Mes Services/Nouveau Service/Onglet DataBases/Lancer Posgresql/onglet Configuration Postgresql-cnpg/onglet Image Catalog/choisissez "postgis-standard-trixie"** puis **Lancer le service**.
2. Au sein du programme `connexion_db.R`, modifier les variables "`name_database`", "`user_name`", "`password`", "`url`", "`port`" pour les adapter au service que vous venez de créer. Les informations nécessaire sà la connexion au serveur PostGre se trouvent sur le bouton `README` du service.
3. Faire tourner tout le programme `create_db.R`. Pour que le programme fonctionne il vous faudra probablement **modifier le chemin de la fonction `source()`**. Après avoir bien fait correctement tourner le programme, vider votre environnement avec la commande `rm(list = ls())`

Exercice 2 - Requêtes sur serveur PostGre

Le début de l'exercice utilise le package DBI. Vous pouvez trouver de la documentation sur ce package ici

1. Connectez-vous au serveur postgre (la syntaxe se trouve au sein d'un programme précédemment lancé). Afficher ensuite les tables présentes sur le serveur avec la fonction `DBI::dbListTables()`.

```
library(DBI)
source("connexion_db_bsaintilan.R", encoding = "utf-8") # à modifier (ici, chemin = à partir du qmd)
conn <- connecter()
DBI::dbListTables(conn)
```

```
 [1] "bpe21_01"          "bpe21_02"          "bpe21_03"
 [4] "bpe21_04"          "bpe21_metro"       "popnaiss_com"
```

```
 [7] "regions_metro"     "spatial_ref_sys"   "geography_columns"
[10] "geometry_columns"
```

2. Afficher les variables présentes dans la table "popnaiss_com". Pour cela vous utiliserez DBI::dbListFields()

```r
DBI::dbListFields(conn,"popnaiss_com") # afficher les variables de la table
```

```
 [1] "codgeo"     "nivgeo"     "libgeo"     "pop"        "sexe_h"
 [6] "sexe_f"     "age_00"     "age_03"     "age_06"     "age_11"
[11] "age_18"     "age_25"     "age_40"     "age_55"     "age_65"
[16] "age_80"     "naisd18_aj"
```

3. Créer une copie de la table popnaiss_com qui sera stockée dans votre environnement. La requête SQL est la suivante 'SELECT * FROM popnaiss_com. Utiliser la fonction dbSendQuery(). Quelle est la nature de l'objet retourné ?

```r
popnaiss <- DBI::dbSendQuery(conn, "SELECT * FROM popnaiss_com;")
str(popnaiss) # l'objet est un résultat de requête SQL 'PqResult'
```

```
Formal class 'PqResult' [package "RPostgres"] with 4 slots
  ..@ conn  :Formal class 'PqConnection' [package "RPostgres"] with 5 slots
  .. .. ..@ ptr         :<externalptr>
  .. .. ..@ bigint      : chr "integer64"
  .. .. ..@ timezone    : chr "UTC"
  .. .. ..@ timezone_out: chr "UTC"
  .. .. ..@ typnames    :'data.frame':  659 obs. of  2 variables:
  .. .. .. ..$ oid    : int [1:659] 16 17 18 19 20 21 22 23 24 25 ...
  .. .. .. ..$ typname: chr [1:659] "bool" "bytea" "char" "name" ...
  ..@ ptr   :<externalptr>
  ..@ sql   : chr "SELECT * FROM popnaiss_com;"
  ..@ bigint: chr "integer64"
```

4. Faire la même question que précedemment en utilisant cette fois la fonction dbGetQuery(). Quelle différence observez-vous ?

```r
popnaiss_get <- DBI::dbGetQuery(conn, "SELECT * FROM popnaiss_com;")
```

```
Warning: Closing open result set, cancelling previous query
```

```r
str(popnaiss_get)
```

```
'data.frame':   34867 obs. of  17 variables:
 $ codgeo    : chr  "01001" "01002" "01004" "01005" ...
 $ nivgeo    : chr  "COM" "COM" "COM" "COM" ...
 $ libgeo    : chr  "L'Abergement-Clémenciat" "L'Abergement-de-Varey" "Ambérieu-en-
Bugey" "Ambérieux-en-Dombes" ...
 $ pop       : num  771 253 14204 1720 112 ...
 $ sexe_h    : num  393.9 128.1 6856.4 861.9 63.1 ...
 $ sexe_f    : num  377.1 124.9 7347.6 858.1 48.9 ...
 $ age_00    : num  28.68 14.58 578.52 60.81 3.05 ...
 $ age_03    : num  17.72 10.41 538.8 66.6 1.02 ...
 $ age_06    : num  56.16 14.58 929.47 120.65 3.05 ...
 $ age_11    : num  88.74 24.99 1258.75 151.54 8.15 ...
 $ age_18    : num  35.91 8.33 1326.92 97.49 7.13 ...
 $ age_25    : num  105.9 56.2 3059.2 352.3 15.3 ...
 $ age_40    : num  193.5 52.1 2527.2 360 28.5 ...
 $ age_55    : num  97.2 26 1552.6 232.6 19.3 ...
 $ age_65    : num  114.2 34.4 1668.4 204.6 18.3 ...
 $ age_80    : num  32.99 11.45 764.09 73.36 8.15 ...
 $ naisd18_aj: num  10 1 229 21 0 33 6 2 8 8 ...
```

```r
#  Avec la fonction dbGetQuery(), on retrouve un dataframe en output.

# Les objets en sortie de la fonction dbSendQuery() sont des objets d'un type particulier.
# Pour ramener cet objet à un dataframe, vous pouvez utiliser la fonction DBI::dbFetch().
```

```r
# Vous pouvez d'ailleurs décider de ne conserver que quelques lignes à votre dataframe
#  (utile quand on manipule de très grosses bases, afin de ne pas saturer l'environnement).
#  Par exemple :
popnaiss <- DBI::dbSendQuery(conn, "SELECT * FROM popnaiss_com;")
popnaiss_5rows <- DBI::dbFetch(popnaiss,n=5) # ne conserve que les 5 premières lignes
# la commande DBI::dbClearResult permet ensuite de libérer les ressources locales et
#  distantes liées au résultat d'une requête.
DBI::dbClearResult(popnaiss)
```

5. En utilisant à nouveau une requête SQL (donc sans travailler sur la copie), afficher l'ensemble des informations concernant la ville de Rennes (`CODGEO = "35238"`) pour la table `popnaiss_com`.

```r
DBI::dbListFields(conn,"popnaiss_com") # afficher les variables de la table
```

```
 [1] "codgeo"     "nivgeo"     "libgeo"     "pop"        "sexe_h"
 [6] "sexe_f"     "age_00"     "age_03"     "age_06"     "age_11"
[11] "age_18"     "age_25"     "age_40"     "age_55"     "age_65"
[16] "age_80"     "naisd18_aj"
```

```r
question5 <- dbGetQuery(conn, statement = "SELECT * FROM popnaiss_com WHERE codgeo='35238';")
question5
```

```
  codgeo nivgeo libgeo    pop   sexe_h   sexe_f   age_00   age_03   age_06
1  35238    COM Rennes 217728 103589.9 114138.1 6945.382 6414.672 10235.76
    age_11  age_18   age_25   age_40   age_55   age_65  age_80 naisd18_aj
1 13865.88 44830.8 50272.82 33494.96 19560.05 20836.38 11271.3       2659
```

6. On souhaite récupérer les informations relatives aux équipements (table bpe_metro) et aux naissances pour la ville de Bruz (table `popnaiss_com`). Pour cela, faites une jointure (en langage SQL) sur la commune de Bruz (`CODGEO = "35047"`). Vous utiliserez l'instruction `INNER JOIN` en terminant avec l'instruction `WHERE`. Indication ici. La clé de jointure est la variable `codgeo` dans `popnaiss_com` et `depcom` dans `bpe21_metro`.

```r
DBI::dbListFields(conn,"bpe21_metro")
```

```
 [1] "id"         "aav2020"    "an"         "bv2012"     "dep"
 [6] "depcom"     "dom"        "epci"       "dciris"     "qp"
[11] "quali_iris" "quali_qp"   "quali_qva"  "quali_zfu"  "quali_zus"
[16] "qualite_xy" "qva"        "reg"        "sdom"       "typequ"
[21] "uu2020"     "zfu"        "zus"        "geometry"
```

```r
# Jointure sur depcom

query <- paste0("SELECT * FROM popnaiss_com INNER JOIN bpe21_metro ON ",
                "popnaiss_com.codgeo=bpe21_metro.depcom WHERE codgeo='35047';")
question6<-dbGetQuery(conn,query)
str(question6)
```

```
'data.frame':   495 obs. of  41 variables:
 $ codgeo    : chr  "35047" "35047" "35047" "35047" ...
 $ nivgeo    : chr  "COM" "COM" "COM" "COM" ...
 $ libgeo    : chr  "Bruz" "Bruz" "Bruz" "Bruz" ...
 $ pop       : num  18516 18516 18516 18516 18516 ...
 $ sexe_h    : num  9078 9078 9078 9078 9078 ...
 $ sexe_f    : num  9438 9438 9438 9438 9438 ...
 $ age_00    : num  533 533 533 533 533 ...
 $ age_03    : num  632 632 632 632 632 ...
 $ age_06    : num  1103 1103 1103 1103 1103 ...
 $ age_11    : num  1882 1882 1882 1882 1882 ...
 $ age_18    : num  2414 2414 2414 2414 2414 ...
 $ age_25    : num  2983 2983 2983 2983 2983 ...
 $ age_40    : num  4056 4056 4056 4056 4056 ...
 $ age_55    : num  2200 2200 2200 2200 2200 ...
 $ age_65    : num  1988 1988 1988 1988 1988 ...
 $ age_80    : num  724 724 724 724 724 ...
```

```
 $ naisd18_aj: num  165 165 165 165 165 165 165 165 165 165 ...
 $ id        : int  9010 9011 345631 17837 161178 161181 161182 161191 161194 161197 ...
 $ aav2020   : chr  "013" "013" "013" "013" ...
 $ an        : chr  "2021" "2021" "2021" "2021" ...
 $ bv2012    : chr  "35238" "35238" "35238" "35238" ...
 $ dep       : chr  "35" "35" "35" "35" ...
 $ depcom    : chr  "35047" "35047" "35047" "35047" ...
 $ dom       : chr  "G" "G" "A" "G" ...
 $ epci      : chr  "243500139" "243500139" "243500139" "243500139" ...
 $ dciris    : chr  "350470105" "350470104" "350470101" "350470106" ...
 $ qp        : chr  "CSZ" "CSZ" "CSZ" "CSZ" ...
 $ quali_iris: chr  "1" "1" "1" "1" ...
 $ quali_qp  : chr  "X" "X" "X" "X" ...
 $ quali_qva : chr  "X" "X" "X" "X" ...
 $ quali_zfu : chr  "X" "X" "X" "X" ...
 $ quali_zus : chr  "X" "X" "X" "X" ...
 $ qualite_xy: chr  "Bonne" "Bonne" "Bonne" "Acceptable" ...
 $ qva       : chr  "CSZ" "CSZ" "CSZ" "CSZ" ...
 $ reg       : chr  "53" "53" "53" "53" ...
 $ sdom      : chr  "G1" "G1" "A4" "G1" ...
 $ typequ    : chr  "G102" "G102" "A401" "G103" ...
 $ uu2020    : chr  "35701" "35701" "35701" "35701" ...
 $ zfu       : chr  "CSZ" "CSZ" "CSZ" "CSZ" ...
 $ zus       : chr  "CSZ" "CSZ" "CSZ" "CSZ" ...
 $ geometry  : 'pq_geometry' chr  "01010000206A08000048E17A147E2E1541295C8FD25EE05941" "01010000206A0
```

7. Le package `dbplyr` permet de travailler sur des BDD de données de type SQL sans avoir à passer par la requête SQL (peut être utile quand on a complètement oublié ses cours… !). Vous trouverez de la documentation ici. Refaire les questions 5 et 6 en utilisant les packages `dbplyr` et `dplyr`. Que fait la fonction `show_query()` ?

```r
library(dplyr)
library(dbplyr)

# Connexion à la table popnaiss
popnaiss <- tbl(conn,"popnaiss_com")
# str(popnaiss) # ce n'est pas encore un data.frame à cette étape !

# Question 5
popnaiss %>%
  filter(codgeo=="35047") %>%
  show_query()
```

```
<SQL>
SELECT "popnaiss_com".*
FROM "popnaiss_com"
WHERE ("codgeo" = '35047')
```

```r
# Pour récupérer les données => utiliser collect()
pop_bruz <- popnaiss %>%
  filter(codgeo=="35047") %>%
  collect()
# str(pop_bruz)

# Question 6
popnaiss %>%
  filter(codgeo=="35047") %>%
  inner_join(
    tbl(conn,"bpe21_metro"),
    by = c("codgeo" = "depcom")
  ) %>%
  show_query() # affiche la requête SQL correspondante
```

```
<SQL>
```

```
SELECT
  "LHS".*,
  "id",
  "aav2020",
  "an",
  "bv2012",
  "dep",
  "dom",
  "epci",
  "dciris",
  "qp",
  "quali_iris",
  "quali_qp",
  "quali_qva",
  "quali_zfu",
  "quali_zus",
  "qualite_xy",
  "qva",
  "reg",
  "sdom",
  "typequ",
  "uu2020",
  "zfu",
  "zus",
  "geometry"
FROM (
  SELECT "popnaiss_com".*
  FROM "popnaiss_com"
  WHERE ("codgeo" = '35047')
) AS "LHS"
INNER JOIN "bpe21_metro"
  ON ("LHS"."codgeo" = "bpe21_metro"."depcom")
```

```r
# Récupération des données
bpe_pop_bruz <- popnaiss %>%
  filter(codgeo=="35047") %>%
  inner_join(
    tbl(conn,"bpe21_metro"),
    by = c("codgeo" = "depcom")
  ) %>%
  collect() # obtenir l'output de la requête : le data.frame

# str(bpe_pop_bruz)
```

Exercice 3 - Manipulation de la Base Permanente des Equipements (BPE)

Au-delà de la découverte de la source BPE, le but de cet exercice est aussi de poursuivre la manipulation de données via les requêtes SQL. Au cours de cet exercice, n'hésitez-pas à comparer les temps de traitement (avec la fonction `system.time()`) entre une solution avec requête SQL et une solution avec requête plus classique (avec `dplyr` par exemple). SPOILER ALERT : cela va + vite en SQL !

1. À partir de la table `bpe21_metro`, créer une table `bpe_dep50` (dans votre environnement) pour le département de la Manche (`DEP='50'`). Cette table contiendra uniquement les variables `ID, DEPCOM, DOM, SDOM, TYPEQU, GEOMETRY`. Quel est le type d'objet retourné ?

```r
bpe_dep50 <- dbGetQuery(conn,
                        statement = paste0("SELECT ID, DEPCOM, DOM, SDOM, TYPEQU, ",
                                           "GEOMETRY FROM bpe21_metro WHERE DEP='50';"))
str(bpe_dep50)
```

```
'data.frame':   15625 obs. of  6 variables:
 $ id      : int  6410 6411 6412 6413 6414 6415 6416 6417 6418 6419 ...
 $ depcom  : chr  "50003" "50041" "50025" "50025" ...
 $ dom     : chr  "G" "G" "G" "G" ...
```

```
 $ sdom    : chr  "G1" "G1" "G1" "G1" ...
 $ typequ  : chr  "G102" "G102" "G102" "G102" ...
 $ geometry: 'pq_geometry' chr  "01010000206A0800008FC2F528D4361641EC51B84E644B5A41" "01010000206A08
```

```
# Les données sont récupérées sous forme d'un dataframe dont la géométrie n'est
#   considérée comme une variable comme une autre
```

2. Refaire la question précédente en utilisant la fonction `st_read()`. La fonction `st_read()` se comporte comme une fonction du package `DBI` sauf que l'argument `statement=` avec les fonctions `DBI` s'appelle `query=` avec `st_read()`.

```r
library(sf)
library(dplyr)
bpe_dep50 <- st_read(conn,
                     query = paste0("SELECT ID, DEPCOM, DOM, SDOM, TYPEQU, GEOMETRY ",
                                    "FROM bpe21_metro WHERE DEP='50';"))
str(bpe_dep50) # 15 625 obs = 15 625 équipements dans le département 50
```

```
Classes 'sf' and 'data.frame':  15625 obs. of  6 variables:
 $ id      : int  6410 6411 6412 6413 6414 6415 6416 6417 6418 6419 ...
 $ depcom  : chr  "50003" "50041" "50025" "50025" ...
 $ dom     : chr  "G" "G" "G" "G" ...
 $ sdom    : chr  "G1" "G1" "G1" "G1" ...
 $ typequ  : chr  "G102" "G102" "G102" "G102" ...
 $ geometry:sfc_POINT of length 15625; first list element:  'XY' num  363957 6892945
 - attr(*, "sf_column")= chr "geometry"
 - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",..: NA NA NA NA NA
  ..- attr(*, "names")= chr [1:5] "id" "depcom" "dom" "sdom" ...
```

```r
# il s'agit bien d'un objet "sf" et "data.frame" : la variable de géométrie est bien
#  prise en compte, non plus comme une variable CHR mais une géométrie
#  (sfc_POINT : données spatiables ponctuelles). Les fonctions sf déjà vues dans les
#  autres TP seront fonctionnelles.

# Remarque importante: Bien entendu, il est possible d'importer la table entière
#  et de la manipuler sur R avec sf et dplyr mais au risque d'une perte de performance.
# Comparons sur la bpe21_metro (qui n'est pas très grosse) et sur une opération de filtrage simple
system.time({
  bpe_dep50 <- st_read(conn,
                       query = paste0("SELECT ID, DEPCOM, DOM, SDOM, TYPEQU, ",
                                      "GEOMETRY FROM bpe21_metro WHERE DEP='50';"))
})
```

```
   user  system elapsed
  0.107   0.014   0.226
```

```r
# user  system elapsed
# 0.142   0.003   0.260

system.time({
  bpe_dep50 <- st_read(conn, query = "SELECT * FROM bpe21_metro;") %>%
    filter(dep == "50") %>%
    select(id, depcom, dom, sdom, typequ)
})
```

```
   user  system elapsed
 37.774   4.797  43.048
```

```r
# user  system elapsed
# 32.265   1.887  34.132
# Ce qui est long ici c'est notamment le temps d'importation de la base complète (on regarde system)
```

3. Quel est le système de projection associé à la table de la question précédente ? Comparer ce système de projection avec celui de la table de la Réunion (`bpe21_04`). Vous identifierez le système de projection de la table de la Réunion de 2 manières différentes. Indice : pour une des façons, vous pouvez faire une requête SQL utilisant la fonction PostGIS `ST_SRID()`. Cette fonction prend en paramètre la variable `GEOMETRY`.

```r
# CRS de la table précédente : CRS 2154 Lambert-93 (classique pour données France Metropolitaine)
st_crs(bpe_dep50)

# CRS de la Réunion
# 1ere façon avec la fonction sf::st_crs
st_read(conn, query = "SELECT * FROM bpe21_04") %>% st_crs() # EPSG 2975 (UTM40S)
#2nde façon avec la fonction POSTGIS ST_SRID(colonne géométrie) => renvoie le crs de chaque élément
dbGetQuery(conn, "SELECT DISTINCT(ST_SRID(geometry)) FROM bpe21_04;") # renvoie tibble 1*1 "2975"
```

4. Dénombrer les maternités (`TYPEQU='D107'`) par region et trier par ordre décroissant. Vous utiliserez la table `bpe21_metro` et les variable `REG` et `TYPEQU` en particulier. Chaque équipement possède un identifiant (`ID`). Pour compter les maternités, vous devrez donc compter les identifiants. Faites cela soit en utilisant `sf+dplyr` soit en utilisant une requête SQL.

```r
# 1ere façon de faire: charger un sf et calculer avec le tidyverse
system.time({
  res1 <- sf::st_read(conn, query = "SELECT * FROM bpe21_metro WHERE TYPEQU='D107';") %>%
    group_by(reg) %>%
    summarise(n_mat = n()) %>%
    arrange(n_mat) %>%
    st_drop_geometry()
})
```

```
   user  system elapsed
  0.139   0.009   0.323
```

```r
#    user  system elapsed
# 0.635   0.026   0.883

# 2nd façon: tout faire en SQL
system.time({
  res2 <- dbGetQuery(conn,
                     statement = paste0("SELECT REG, COUNT(id) as n_mat FROM bpe21_metro ",
                                        "WHERE TYPEQU='D107' GROUP BY REG ORDER BY n_mat;"))
})
```

```
   user  system elapsed
  0.004   0.000   0.189
```

```r
# user  system elapsed
# 0.041   0.009   0.298
```

5. Changement de thème : on s'intéresse aux cinémas ! Sélectionner les cinémas (`TYPEQU='F303'`) dans un rayon d'un 1km autour de la Sorbonne (5e arrondissemnt de Paris). On pourra utiliser les coordoonnées (`long`,`lat`) suivantes (`lat = 48.84864, long = 2.34297`) pour situer La Sorbonne. Attention, les coordonnées de longitude et de lattitude font référence au système WGS84 (CRS 4326).

a. Créer la table "`cinema_bpe`" à partir de la table `bpe21_metro`. Vous utiliserez une requête SQL ou une syntaxe `sf+dplyr`.

```r
# 1ere façon: principalement avec sf
# On filtre la bpe en SQL
cinemas_bpe <- sf::st_read(conn, query = "SELECT * FROM bpe21_metro WHERE TYPEQU='F303';")
str(cinemas_bpe)
```

```
Classes 'sf' and 'data.frame':  1936 obs. of  24 variables:
 $ id        : int  1996872 1996881 1996893 1996900 1996931 1994277 1994281 1994287 1994288 1994289
 $ aav2020   : chr  "045" "539" "348" "100" ...
 $ an        : chr  "2021" "2021" "2021" "2021" ...
 $ bv2012    : chr  "25434" "36006" "36018" "36031" ...
 $ dep       : chr  "25" "36" "36" "36" ...
 $ depcom    : chr  "25434" "36006" "36018" "36031" ...
 $ dom       : chr  "F" "F" "F" "F" ...
 $ epci      : chr  "200068070" "200068872" "243600319" "243600301" ...
```

```
 $ dciris    : chr  "254340000" "360060000" "360180000" "360310000" ...
 $ qp        : chr  "CSZ" "CSZ" "CSZ" "CSZ" ...
 $ quali_iris: chr  "X" "X" "X" "X" ...
 $ quali_qp  : chr  "X" "X" "X" "X" ...
 $ quali_qva : chr  "X" "X" "X" "X" ...
 $ quali_zfu : chr  "X" "X" "X" "X" ...
 $ quali_zus : chr  "X" "X" "X" "X" ...
 $ qualite_xy: chr  "Bonne" "Bonne" "Bonne" "Bonne" ...
 $ qva       : chr  "CSZ" "CSZ" "CSZ" "CSZ" ...
 $ reg       : chr  "27" "24" "24" "24" ...
 $ sdom      : chr  "F3" "F3" "F3" "F3" ...
 $ typequ    : chr  "F303" "F303" "F303" "F303" ...
 $ uu2020    : chr  "25110" "36203" "36201" "36109" ...
 $ zfu       : chr  "CSZ" "CSZ" "CSZ" "CSZ" ...
 $ zus       : chr  "CSZ" "CSZ" "CSZ" "CSZ" ...
 $ geometry  :sfc_POINT of length 1936; first list element:  'XY' num  938401 6672153
 - attr(*, "sf_column")= chr "geometry"
 - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",..: NA NA NA NA NA NA NA NA NA NA ...
  ..- attr(*, "names")= chr [1:23] "id" "aav2020" "an" "bv2012" ...
```

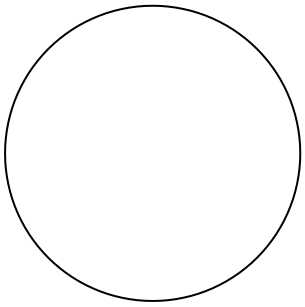
```r
# Le reste des opérations notamment les opérations géométriques sont réalisés avec sf sur R
```

b. Executez le code suivant. Ce code crée un “buffer” (périmètre) autour de la Sorbonne.

```r
# On construit un buffer de 1km (une zone tampon) autour de la Sorbonne
  # df des coordonnées
sorbonne_buffer <- data.frame(x=2.34297,y=48.84864) %>%
  #qu'on transforme en objet sf (systeme de proj WGS84 => crs=4326)
    st_as_sf(coords = c("x","y"), crs = 4326) %>%
  # on reprojette en LAMBERT-93 (crs=2154)
  st_transform(2154) %>%
  # on crée la zone tampon autour du point (l'unité est le mètre ici)
  st_buffer(1000)

str(sorbonne_buffer) # le buffer est constitué d'un unique polygône
```

```
Classes 'sf' and 'data.frame':  1 obs. of  1 variable:
 $ geometry:sfc_POLYGON of length 1; first list element: List of 1
  ..$ : num [1:121, 1:2] 652784 652783 652779 652772 652763 ...
  ..- attr(*, "class")= chr [1:3] "XY" "POLYGON" "sfg"
 - attr(*, "sf_column")= chr "geometry"
 - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",..:
  ..- attr(*, "names")= chr(0)
```

```r
plot(sorbonne_buffer %>% st_geometry()) # qui s'avère être un cercle
```



c. À partir de la table `cinema_bpe`, récupérer l'ensemble des cinémas contenus dans le buffer. Vous pouvez utiliser la fonction `st_within()` de `sf`.

```
# On détermine si chaque cinéma de la bpe  appartient ou non au buffer avec la fonction st_within
cinema_1km_sorbonne_list <- st_within(cinemas_bpe, sorbonne_buffer) # on obtient une liste
 # str(cinema_1km_sorbonne_list)
cinema_1km_sorbonne <- cinemas_bpe %>% filter(lengths(cinema_1km_sorbonne_list)>0)
cinema_1km_sorbonne %>% nrow() #21 cinémas
```

```
[1] 21
```

```
# Ou
cinema_1km_sorbonne<-st_intersection(sorbonne_buffer,cinemas_bpe)
```

```
Warning: attribute variables are assumed to be spatially constant throughout
all geometries
```

```
# 2nde façon: travailler en SQL avec POSTGIS

# Le système de projection d'une base postgis peut se retrouver avec la fonction Find_SRID
# qui prend trois arguments: le nom du schéma (ici public), le nom de la table et le nom de
#  la colonne de la table correspondant à la géométrie.
crs_bpe <- pull(dbGetQuery(conn, "SELECT Find_SRID('public','bpe21_metro', 'geometry');"))

# Ici on remarque que le crs est 2154 (Lambert-93).
# Il faut donc harmoniser les coordonnées du point Sorbonne et les projeter en Lambert-93
# Pour cela on va créer un objet spatial en postgis de type POINT à partir des coordonnées
#  WGS84 (epsg = 4326) fournies
sorbonne <- "ST_GeomFromText('POINT(2.34297 48.84864)', 4326)"
# et le reprojeter en Lambert-93.
sorbonne <- paste0("ST_Transform(", sorbonne, ", 2154)")
# Autour de la Sorbonne on crée un buffer cad une zone tampon (ici un disque de diamètre 1km)
sorbonne_buffer_req <- paste0("ST_Buffer(", sorbonne ,", 1000)")

# On peut dès lors écrire la requête avec l'instruction ST_WITHIN
# ST_within(A,B) indique si une géométrie A (ici nos points de la BPE) appartient
#  à une géométrie B (ici notre buffer)
query <- paste0(
  "SELECT bpe.* FROM bpe21_metro as bpe, ", sorbonne_buffer_req, " AS sorbuff
  WHERE ST_Within(bpe.geometry, sorbuff.geometry) and TYPEQU='F303';"
)
cinema_1km_sorbonne <- sf::st_read(conn, query = query)
#on utilise sf::st_read pour récupérer un objet spatial R plutôt qu'un dataframe
```

```
# str(cinema_1km_sorbonne) :  'sf' and 'data.frame':  21 obs. of  24 variables

nrow(cinema_1km_sorbonne) # 21 cinémas dans un rayon de 1km (à vol d'oiseau)
```

```
[1] 21
```

6. Représenter ces cinémas sur une carte `leaflet` en utilisant le code suivant. Les cinémas repérés sont-ils tous bien contenus dans le buffer ? Voyez-vous des cinémas qui n'auraient pas été filtrés ?

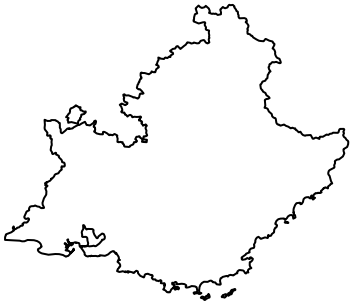
```
library(leaflet)

# Optionnel :
# On récupère une icone spécifique sur https://ionic.io/ionicons (mot clé film)
#  cinemaIcons <- makeIcon(iconUrl = "images/film-sharp.png", 18,18)

leaflet() %>%
  setView(lat = 48.84864, lng = 2.34297, zoom = 15) %>%
  addTiles() %>%
  addMarkers(lat = 48.84864, lng = 2.34297) %>%
  addCircles(
    lat = 48.84864, lng = 2.34297, weight = 1, radius = 1000
  ) %>%
  addMarkers(data = cinema_1km_sorbonne %>% st_transform(4326))
# On peut vérifier que les infos bpe se superposent très bien aux infos OSM


# Remarque : 1000m en LAMBERT-93 ce n'est pas exactement 1000m en WGS84
#  (zoomez sur la carte suivante)
leaflet() %>%
  setView(lat = 48.84864, lng = 2.34297, zoom = 15) %>%
  addTiles() %>%
  addCircles(
    lat = 48.84864, lng = 2.34297, weight = 1, radius = 1000
  ) %>%
  addPolygons(data=sorbonne_buffer %>% st_transform(4326), col = "red")
# Les 1000m en LAMBERT-93 ne sont pas exactement 1000m en WGS84 (1000m "réels")
# On aurait pu donc projeter la bpe en wgs84 pour tenir compte de cet écart.
```

Exercice 4 - Illustration d'un problème fréquent de géolocalisation : les boulodromes de PACA !

Le but de cet exercice est de vous sensibiliser aux problèmes de cohérence pouvant exister entre deux sources géolocalisées. Bien que projeté dans un même CRS, les données peuvent ne pas correspondre. Cela est particulièrement fréquent lorsqu'on travaille avec des polygones simplifiés. Illustration :

1. À partir de la table `bpe21_metro`, récupérer l'ensemble des boulodromes (`TYPEQU="F102"`) de France. Vous stockerez cela au sein d'une table "`boulodrome`". A partir de la table `regions_metro`, créer une table "`paca`" qui ne contiendra que la région PACA (`code='93'`).

```
paca <- st_read(conn, query = "SELECT * FROM regions_metro WHERE code = '93';")
plot(paca %>% st_geometry())
```

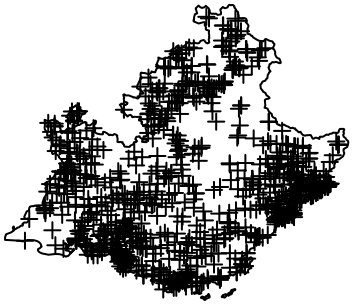


```r
boulodromes <- st_read(conn,
                       query = "SELECT id, typequ, geometry FROM bpe21_metro WHERE typequ = 'F102';")
str(boulodromes)
```

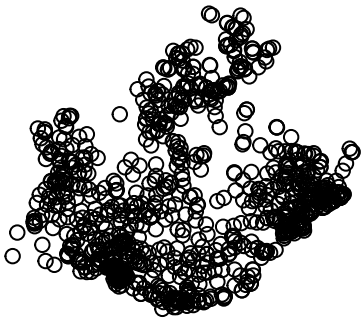
```
Classes 'sf' and 'data.frame':  21187 obs. of  3 variables:
 $ id      : int  1765527 1765534 1765539 1765553 1765601 1765606 1765618 1765635 1765639 1765640 ..
 $ typequ  : chr  "F102" "F102" "F102" "F102" ...
 $ geometry:sfc_POINT of length 21187; first list element:  'XY' num  688286 6840223
 - attr(*, "sf_column")= chr "geometry"
 - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",..: NA NA
  ..- attr(*, "names")= chr [1:2] "id" "typequ"
```

2. Créer une table `boulodrome_paca` en utilisant la geometry des 2 tables. L'idée est de faire une intersection des deux fonds. Vous ferez cette intersection au choix : soit en utilisant `sf` soit en utilisant PostGIS. Dans les deux cas, la fonction s'appelle `st_contains()`.

```r
# 1ere façon: réaliser les opérations géométriques avec R grâce au package sf
boulodromes_paca_list <- st_contains(paca, boulodromes)
boulodromes_paca <- boulodromes %>% slice(boulodromes_paca_list[[1]])

plot(paca %>% st_geometry())
plot(boulodromes_paca %>% st_geometry(), pch = 3, cex = 0.8, add = TRUE)
```



```
# 2nde façon: réaliser le traitement géométrique avec POSTGIS et la fonction ST_contains
query <- "SELECT bpe.* FROM bpe21_metro AS BPE, regions_metro AS regions
WHERE ST_Contains(regions.geometry, bpe.geometry) and bpe.typequ='F102' and regions.code = '93';"

boulodromes_paca <- st_read(conn, query = query)
plot(boulodromes_paca %>% st_geometry())
```



```
# Cette 2nde façon est à préférer pour des objets trop lourds à charger en mémoire.
```

3. Le nombre de boulodromes en PACA issu de l'intersection correspond-t-il au nombre de boulodromes en PACA issu de la table `bpe21_metro` ? Pour faire cela :

a. Récupérer les boulodromes de PACA en filtrant la `bpe21_metro` sur les variables `typequ = 'F102'`et `dep in ('04','05','06','13','83','84')`.

```
# on peut vérifier le résultat en récupérant directement les boulodromes de PACA depuis la BPE
# Si des différences existent, essayez de comprendre pourquoi.
boulodromes_paca_bis <- st_read(conn,
                                query = paste0("SELECT id, typequ, dep, qualite_xy, geometry ",
                                               "FROM bpe21_metro WHERE typequ = 'F102' and dep ",
```

```
                                                "in ('04','05','06','13','83','84');"))
```

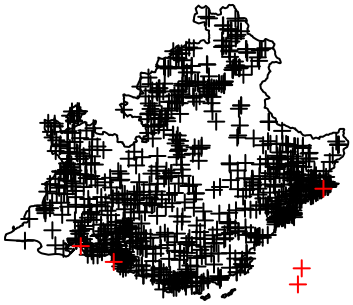
b. Exécuter le code suivant. Identifier les éventuels problèmes.

```r
# les deux data n'ont pas le même nb d'observations (904 vs 910)
diff <- boulodromes_paca_bis %>% mutate(version_bis = TRUE) %>%
  st_join(
    boulodromes_paca %>% mutate(version_orig = TRUE) %>% select(-typequ), by = "id"
  ) %>%
  filter((is.na(version_bis) | (is.na(version_orig))))
```

```r
# on réucpère 6 boulodromes supplémentaires:
diff
```

```
Simple feature collection with 6 features and 28 fields
Geometry type: POINT
Dimension:     XY
Bounding box:  xmin: 860939.5 ymin: 6017007 xmax: 1057403 ymax: 6304539
Projected CRS: RGF93 v1 / Lambert-93
       id.x typequ dep.x qualite_xy.x version_bis id.y aav2020   an bv2012
196 1918692   F102    13        Bonne        TRUE   NA    <NA> <NA>   <NA>
521 1922646   F102    06        Bonne        TRUE   NA    <NA> <NA>   <NA>
534 1922731   F102    06        Bonne        TRUE   NA    <NA> <NA>   <NA>
634 1930525   F102    06        Bonne        TRUE   NA    <NA> <NA>   <NA>
839 1927499   F102    13        Bonne        TRUE   NA    <NA> <NA>   <NA>
894 1931455   F102    06        Bonne        TRUE   NA    <NA> <NA>   <NA>
    dep.y depcom  dom epci dciris   qp quali_iris quali_qp quali_qva quali_zfu
196  <NA>   <NA> <NA> <NA>   <NA> <NA>       <NA>     <NA>      <NA>      <NA>
521  <NA>   <NA> <NA> <NA>   <NA> <NA>       <NA>     <NA>      <NA>      <NA>
534  <NA>   <NA> <NA> <NA>   <NA> <NA>       <NA>     <NA>      <NA>      <NA>
634  <NA>   <NA> <NA> <NA>   <NA> <NA>       <NA>     <NA>      <NA>      <NA>
839  <NA>   <NA> <NA> <NA>   <NA> <NA>       <NA>     <NA>      <NA>      <NA>
894  <NA>   <NA> <NA> <NA>   <NA> <NA>       <NA>     <NA>      <NA>      <NA>
    quali_zus qualite_xy.y  qva  reg sdom uu2020  zfu  zus version_orig
196      <NA>         <NA> <NA> <NA> <NA>   <NA> <NA> <NA>           NA
521      <NA>         <NA> <NA> <NA> <NA>   <NA> <NA> <NA>           NA
534      <NA>         <NA> <NA> <NA> <NA>   <NA> <NA> <NA>           NA
634      <NA>         <NA> <NA> <NA> <NA>   <NA> <NA> <NA>           NA
839      <NA>         <NA> <NA> <NA> <NA>   <NA> <NA> <NA>           NA
894      <NA>         <NA> <NA> <NA> <NA>   <NA> <NA> <NA>           NA
                    geometry
196 POINT (860939.5 6258077)
521  POINT (1036720 6227100)
534  POINT (1057403 6304539)
634  POINT (1015370 6017007)
839 POINT (887624.2 6245292)
894  POINT (1039797 6240096)
```

```r
plot(paca %>% st_geometry())
plot(boulodromes_paca %>% st_geometry(), pch = 3, cex = 0.8, add = TRUE)
plot(diff %>% st_geometry(), col = "red", pch = 3, cex = 0.8, add = TRUE)
```



```
# Pour deux d'entre eux, la géolocalisation de ces boulodromes est absurde car en pleine mer

# Vérifions plus précisément les autres cas avec une carte leaflet
```

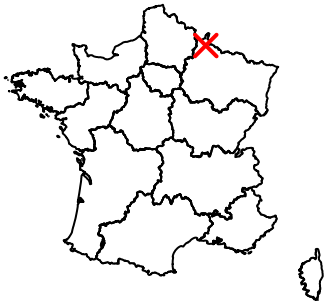
```
leaflet() %>%
  setView(lat = 43.8741, lng = 6.0287, zoom = 8) %>%
  addTiles() %>%
  addMarkers(data = diff %>% st_transform(4326)) %>%
  addPolygons(data = paca %>% st_transform(4326), stroke = 1, color = "red")

# On observe que (lecture d'ouest en est)
# - le polygône paca simplifie le tracé réel de la région => l'extrémité de la commune
#  de Port-de-Bouc est hors du polygône, hors un boulodrome s'y trouve.
# - les îles du Frioul ne sont pas incluses dans le polygône paca
#   => 1 boulodrome est situé sur ces îles
# - 3 boulodromes sont situés sur la mer (dont 1 est très éloigné)
# - 1 dernier également en limite du polygône qui "oublie" une partie du territoire

# Pour des raisons de stockage notamment, les fonds sont souvent simplifiés.
#  L'exercice de géolocalisation n'est pas parfait.
```

Exercice 5 - Optionnel mais pouvant vous servir un jour !

Cet exercice traite des courbes isochrones. En cartographie, ces courbes désignent classiquement des lignes reliant des points situés à des temps de parcours identiques. Elles sont souvent utiles lorsqu'on cherche à identifier l'accessibilité d'un équipement.

1. Choisir un équipement (= une ligne de la table `bpe21_metro`). Récupérer ses coordonnées et les convertir en WGS84 (`epsg=4326`).

```
# Ici, on prend pour exemple les maternités
mater <- sf::st_read(conn, query = "SELECT * FROM bpe21_metro WHERE TYPEQU='D107';") %>%
  slice(1)

# On récupère ses coordonnées
mater_coords <- st_coordinates(mater) %>% as.numeric

sf_reg_metro <- st_read(conn, query = "SELECT * FROM regions_metro")
plot(st_geometry(sf_reg_metro))
points(x = mater_coords[1], y = mater_coords[2], pch = 4, lwd = 2, cex = 1.5, col = "red")
```



```r
# On transforme ses coordonnées en WGS84 (epsg=4326)
mater_coords <- st_coordinates(mater %>% st_transform(4326)) %>% as.numeric
```

2. Situer l'équipement sur une carte avec `leaflet`.

```r
leaflet() %>%
  setView(lng = mater_coords[1], lat = mater_coords[2], zoom = 14) %>%
  addTiles() %>%
  addMarkers(lng = mater_coords[1], lat = mater_coords[2])
```

3. Calculer les isochrones avec `osrm::osrmIsochrone()`.

```r
# install.packages("osrm")
# Attention, cela peut prendre quelques minutes
iso <- osrm::osrmIsochrone(
  loc = mater_coords, # coordonnées du point de référence
  breaks = seq(0,60,10), # valeurs des isochrones à calculer en minutes
  res = 100 # détermine le nombre de points utilisés (res*res) pour dessiner les isochornes
)
```

```
'res' is deprecated, use 'n' instead.
```

```r
str(iso)
```

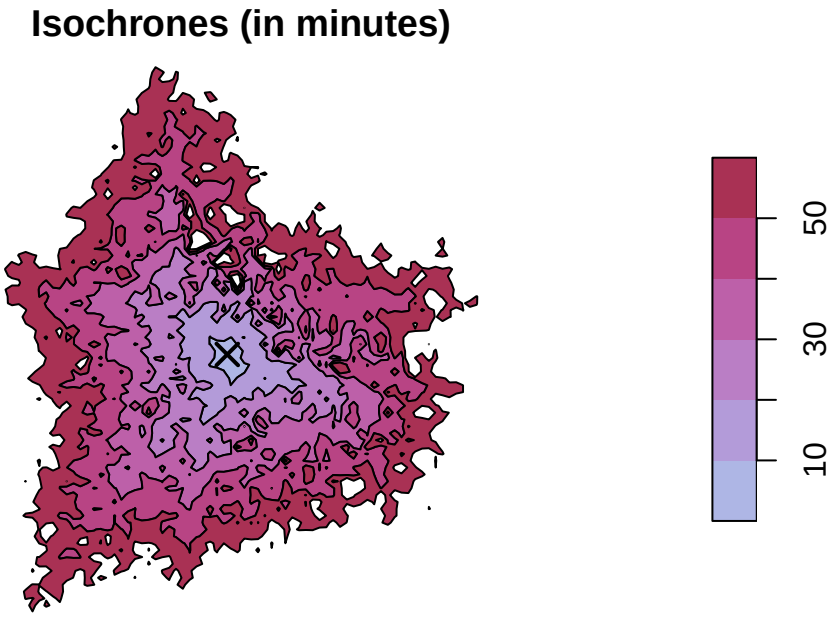
```
Classes 'sf' and 'data.frame':  6 obs. of  4 variables:
 $ id      : int  1 2 3 4 5 6
 $ isomin  : num  0 10 20 30 40 50
 $ isomax  : num  10 20 30 40 50 60
 $ geometry:sfc_MULTIPOLYGON of length 6; first list element: List of 2
  ..$ :List of 1
  .. ..$ : num [1:5, 1:2] 4.75 4.74 4.75 4.75 4.75 ...
  ..$ :List of 1
  .. ..$ : num [1:31, 1:2] 4.64 4.64 4.64 4.64 4.66 ...
  ..- attr(*, "class")= chr [1:3] "XY" "MULTIPOLYGON" "sfg"
 - attr(*, "sf_column")= chr "geometry"
 - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",..: NA NA NA
  ..- attr(*, "names")= chr [1:3] "id" "isomin" "isomax"
```

```r
gc()
```

```
          used (Mb) gc trigger  (Mb)  max used   (Mb)
Ncells 1554433 83.1   10728364 573.0  20953834 1119.1
Vcells 4697906 35.9   85574281 652.9 153407785 1170.5
```

4. Représenter ces isochrones sous forme d'une carte choroplèthe.

```
bks <-  sort(unique(c(iso$isomin, iso$isomax)))
pals <- hcl.colors(n = length(bks) - 1, palette = "Red-Blue", rev = TRUE)
plot(iso["isomax"], breaks = bks, pal = pals,
     main = "Isochrones (in minutes)", reset = FALSE)
points(x = mater_coords[1], y = mater_coords[2], pch = 4, lwd = 2, cex = 1.5)
```




```
leaflet() %>%
  setView(lng = mater_coords[1], lat = mater_coords[2], zoom = 8) %>%
  addTiles() %>%
  addMarkers(lng = mater_coords[1], lat = mater_coords[2]) %>%
  addProviderTiles(
    providers$CartoDB.DarkMatter,
    options = providerTileOptions(opacity = 0.4)) %>%
  addPolygons(
    data=iso,
    fillColor = pals,
    smoothFactor = 0.3,
    weight = 1,
    opacity = 1,
    color = "white",
    dashArray = "3",
    fillOpacity = 0.65
  ) %>%
  addLegend(
    position="bottomleft",
    colors=pals,
    labels=rev(c("50-60","40-50",
                 "30-40","20-30","10-20", "0-10")),
    opacity = 0.6,
    title="Temps de trajet par la route (en minutes)")
```